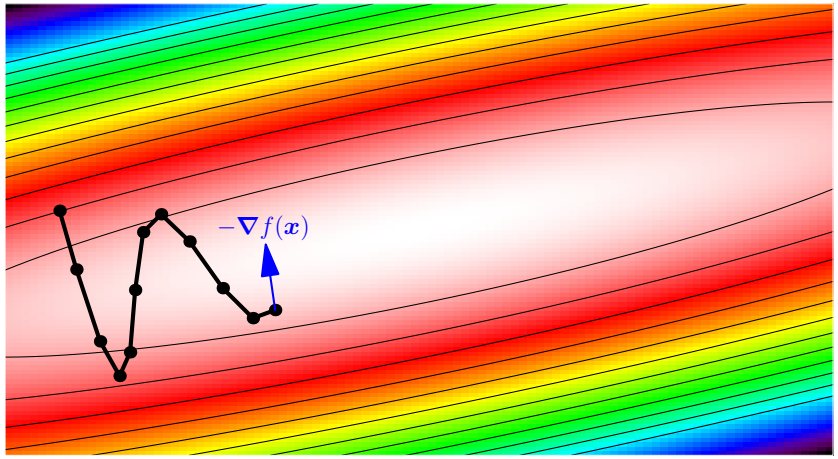
Mathematics of Machine Learning

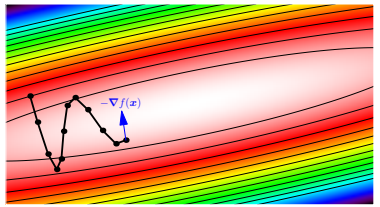
Stochastic Gradient Descent



SGD, momentum, step size, RMSProp, Adam, learning-rate schedules

Outline

1. **SGD**
2. Momentum
3. Loss Landscapes



The Next Step

- In most machine learning problems we design a network and a loss function
- The rest requires simple optimisation
- Although Newton and Quasi-Newton methods ensure rapid convergence for many tasks this apparent advantage is illusory
- Since the rise of deep learning, gradient descent and its variants have become dominant

Mini-Batch Learning

- Rather than computing the gradient of the loss function, e.g.

$$\nabla L_{\mathcal{D}}(\boldsymbol{w}) = \frac{1}{|\mathcal{D}|}\nabla \sum_{(\boldsymbol{x},y)\in\mathcal{D}} L\big(f(\boldsymbol{x}|\boldsymbol{w}),y\big)$$

- We often compute an approximation

$$\nabla L_{\mathcal{B}}(\boldsymbol{w}) = \frac{1}{|\mathcal{B}|}\nabla \sum_{(\boldsymbol{x},y)\in\mathcal{B}} L\big(f(\boldsymbol{x}|\boldsymbol{w}),y\big)$$

- where $\mathcal{B} \subset \mathcal{D}$ is a randomly sampled subset of the training set
- Usually $|\mathcal{B}| \ll |\mathcal{D}|$

Stochastic Gradient Descent

- Stochastic gradient descent (SGD) follows the gradient of mini-batches
- Computationally this is efficient because it is much quicker to compute $\nabla L_{\mathcal{B}}(\boldsymbol{w})$ than $\nabla L_{\mathcal{D}}(\boldsymbol{w})$
- The batch gradients add noise (are stochastic) as we are only sampling the dataset
- This noise might help escape local-minima (but I'm not sure there is compelling evidence for this)
- Taking many smaller steps of approximate gradients reduces the likelihood of divergence

Automatic Differentiation

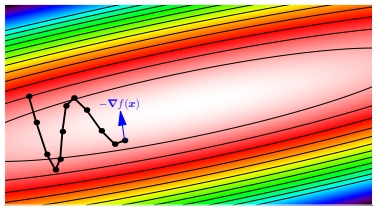
- For gradient descent we need to compute the gradient
- For most of my life this was just a pain
- You guys have it easy, modern deep learning frameworks do this for you automatically
- This is a *game changer*! We are no longer afraid of large models
- We can differentiate through algorithms allowing us to train incredibly sophisticated networks

Convergence

- Asymptotic convergence of gradient descent (and SGD) is slower than quasi-Newton methods
- But there are three reasons why we don't really care
  - ⋆ For complex models we spend a lot of time before we are close to a quadratic minimum where asymptotic convergence kicks in
  - ⋆ These days we often use ReLUs (rectified linear units) which are not differentiable at 0. The convergence we discussed depends on a Taylor expansion that assumes a smooth (twice differentiable) loss
  - ⋆ We want to minimise the generalisation error, but are only minimising a poor surrogate, namely the training error

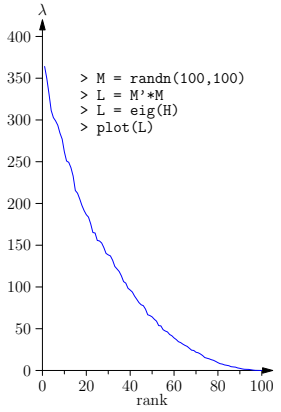
Outline

1. SGD
2. **Momentum**
3. Loss Landscapes



Recap: One Step Size, Many Curvatures

- The Hessian has a spectrum of eigenvalues: some directions are steep, others nearly flat
- One step size r must satisfy $r < 2/\lambda_{\max}$, or we diverge (and quickly get NaN errors)
- The flat directions then shrink by at best $1 - 2/\kappa$ per step
- This seems hopelessly slow, yet plain SGD (nearly always with momentum) is widely used in deep learning



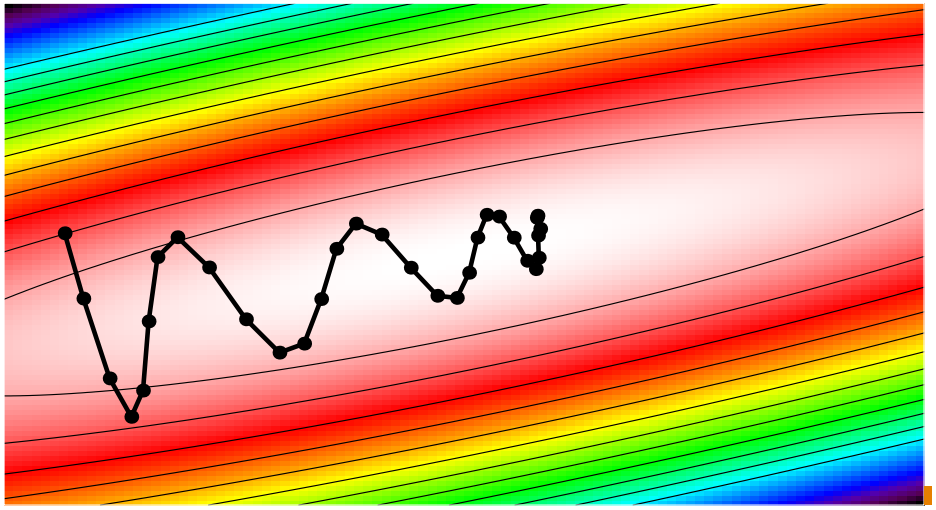
Momentum

- In high dimensions we zig-zag:
  - ⋆ stepping consistently in directions with low curvature
  - ⋆ jumping backwards and forwards past the minimum in directions with high curvature
- By introducing "momentum" we can increase our steps in low curvature directions and decrease them in high curvature directions

$$\boldsymbol{v}(t+1) = \gamma \boldsymbol{v}(t) - r\boldsymbol{\nabla} f(\boldsymbol{x}(t))$$
$$\boldsymbol{x}(t+1) = \boldsymbol{x}(t) + \boldsymbol{v}(t+1)$$

$0 < \gamma < 1$ sets how much velocity is kept ($1 - \gamma$ acts as friction)

Gradient Descent with Momentum



Nesterov Momentum

- Nesterov's variant takes the gradient where momentum is about to carry us

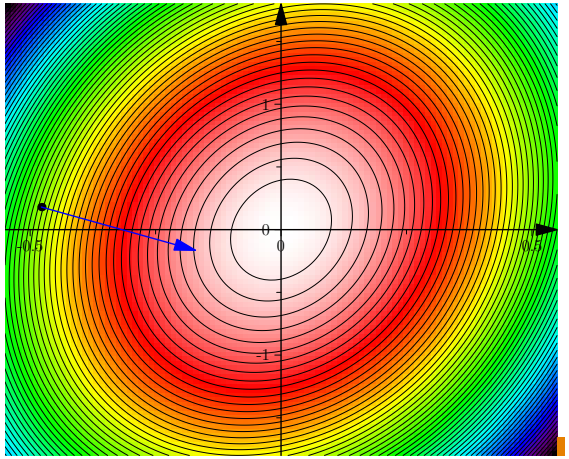
$$\boldsymbol{v}(t+1) = \gamma \boldsymbol{v}(t) - r\boldsymbol{\nabla} f\big(\boldsymbol{x}(t) + \gamma \boldsymbol{v}(t)\big)$$
$$\boldsymbol{x}(t+1) = \boldsymbol{x}(t) + \boldsymbol{v}(t+1)$$

- With well-tuned momentum the number of steps needed scales as $\sqrt{\kappa}$ rather than κ
- The convergence is still **linear**: momentum fixes the condition number, not the order of convergence
- On convex problems Nesterov reaches error $O(1/t^2)$ rather than $O(1/t)$, the best any gradient method can do (much more in COMP6260, Optimisation for Machine Learning)

Adaptive Methods

- A major difficulty of high dimensional optimisation is the existence of different scales
- That is, there are some directions where we have to move a lot (low curvature), while other directions we have to make small steps
- In adaptive methods we use a dynamic algorithm to rescale the step size of each parameter (weight)
- We can think of this as a "regularisation" that makes our basins of attraction more spherical

Rescaling Co-ordinates



RMSProp

- Rescale each coordinate by the typical size of its gradient
- Keep a running average of the squared gradients

$$S_i(t+1) = (1-\gamma)S_i(t) + \gamma\left(\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}\right)^2$$

- and divide the step by its square root

$$w_i(t+1) = w_i(t) - \frac{r}{\sqrt{S_i(t+1)}+\epsilon}\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}$$

- Steep directions take small steps and flat ones large steps: each weight moves about r per step, whatever the scale of the loss

Adding in Momentum

- RMSProp has no momentum (one argument is that rescaling has already "regularised the landscape")
- Nevertheless we can learn a momentum term

$$M_i(t+1) = (1-\beta)M_i(t) + \beta\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}$$

$$S_i(t+1) = (1-\gamma)S_i(t) + \gamma\left(\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}\right)^2$$

- Starting from $M_i(0) = S_i(0) = 0$ biases these averages towards zero at first, which we correct

$$\hat{M}_i(t+1) = \frac{M_i(t+1)}{1-(1-\beta)^{t+1}} \qquad \hat{S}_i(t+1) = \frac{S_i(t+1)}{1-(1-\gamma)^{t+1}}$$

ADAM

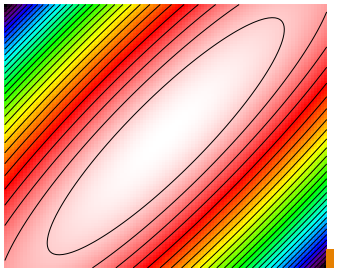
- Adaptive Moment Estimation (Adam) adapts the scale of the parameters, but also uses momentum
- Update weights

$$w_i(t+1) = w_i(t) - \frac{r}{\sqrt{\hat{S}_i(t+1)} + \epsilon}\hat{M}_i(t+1)$$

- Adam and its variants are very successful in deep learning: a network often learns with Adam when it would not with SGD
- **AdamW** applies weight decay directly to the weights, $w_i \leftarrow w_i - r\eta w_i$, rather than through the loss, where Adam's rescaling would weaken it; the default for transformers

Covariant Equations

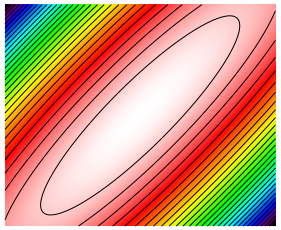
- Vector arithmetic, (matrix multiplication, addition, multiplying by a scale, computing gradients) has a covariant property that it is invariant to the coordinate system
- That is we can translate and rotate our coordinates and get the same result
- That is not true of element-wise multiplication of vectors
- Many ML algorithms do this (including RMSProp and Adam), but they aren't invariant if we rotate our coordinates



Can't rescale coordinates

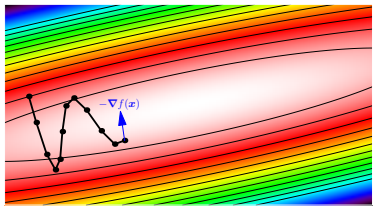
Correlated Weights

- Correlated weights will lead to skewed valleys
- Adaptive methods would be more efficient if they were rotationally invariant
- However, this would slow down the algorithm
- ADAM is a compromise between speed of implementation and effectiveness



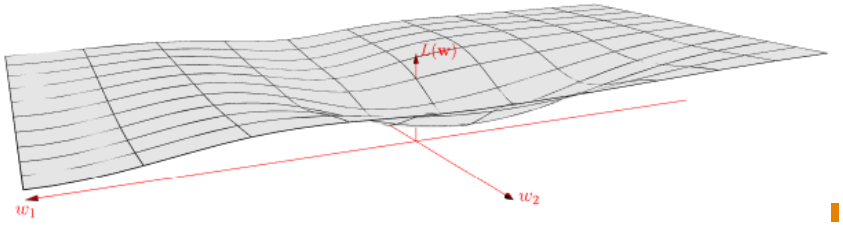
Outline

1. SGD
2. Momentum
3. **Loss Landscapes**



Loss Landscape

- A useful concept for understanding optimisation is the loss landscape
- For every value of the weights $\boldsymbol{w}$ there is some loss associated with it
- Note that this landscape is very high dimensional, so our intuition can be a bit misleading
- The landscape is also huge



Global and Local Minima

- Our objective is to minimise our loss
- This would mean finding the global minimum
- For non-convex losses no efficient algorithm is guaranteed to find the global minimum
- The best we can hope is to find a local minimum by doing gradient descent
- In practice, our landscapes are so large that we are probably unable to find even a local minimum

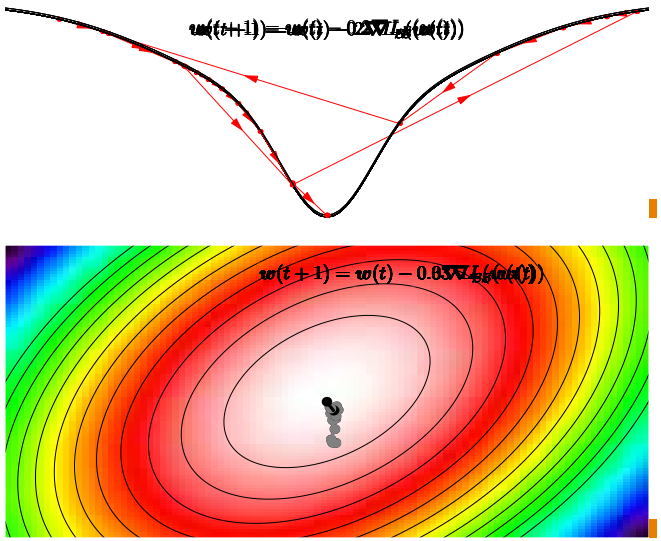
Noisy Optimisation

- Because we are using mini-batches we aren't actually following the correct gradients
- If we reduce our step size then our average gradient is closer to the true gradient
- Thus our optimisation is inherently noisy
- At least for deep learning there is no evidence that the weights stop changing even after learning for a huge amount of time

Valleys in Valleys in Valleys

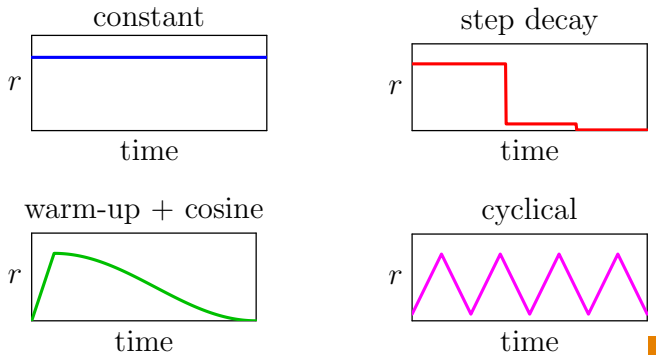
- My view of the loss landscape is that we have valleys inside bigger valleys inside bigger valleys, and so on
- If our noise is high (we use large step size), we can navigate away from local valleys and move towards the big valley
- But, we can't explore the small valleys
- Often when the rate of improvement slows down, practitioners will reduce their learning rate by a factor of 10 and the rate of improvement jumps up
- Some practitioners cycle through raising and lowering their learning rates which may speed up convergence

Reducing the Step Size



Learning-Rate Schedules

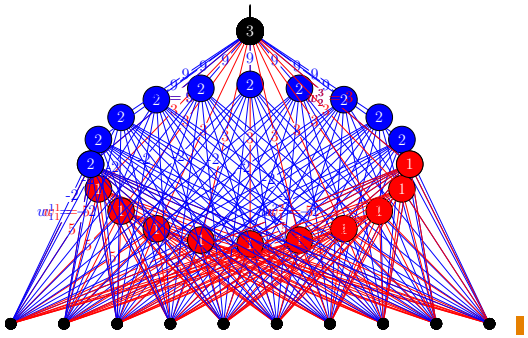
- Large r early: noisy steps cross the small valleys and find the big ones
- Small r late: settle into a valley



- A short warm-up stops the first steps diverging before Adam's running averages have settled; common for transformers

Symmetries

- When training neural networks (deep or shallow) there are typically many symmetries
- Examples come from permuting the neurons (or filters in CNNs) within a layer, together with their weights



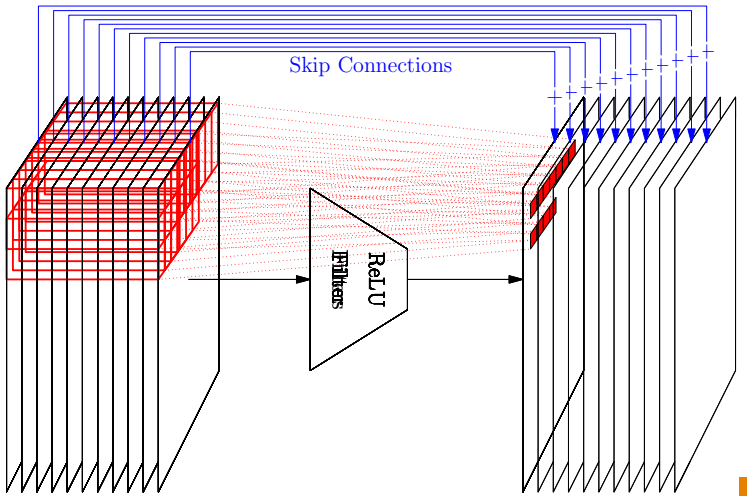
More Symmetries

- With n hidden nodes there are $n!$ symmetries
- For odd activation functions such as $\tanh$, multiplying all the input and output weights of a node by -1 gives the same network
- There are also continuous symmetries. For linear networks with two layers performing a mapping $\boldsymbol{W}_2\boldsymbol{W}_1$ then for any invertible matrix $\mathbf{U}$ (where $\mathbf{U}\mathbf{U}^{-1} = \mathbf{I}$)

$$\boldsymbol{W}_2\boldsymbol{W}_1 = \boldsymbol{W}_2\mathbf{I}\boldsymbol{W}_1 = \boldsymbol{W}_2(\mathbf{U}\mathbf{U}^{-1})\boldsymbol{W}_1 = (\boldsymbol{W}_2\mathbf{U})(\mathbf{U}^{-1}\boldsymbol{W}_1)$$

  so a network with weights $\boldsymbol{W}_2' = \boldsymbol{W}_2\mathbf{U}$ and $\boldsymbol{W}_1' = \mathbf{U}^{-1}\boldsymbol{W}_1$ will perform the same mapping
- There will be a huge space of equivalent networks

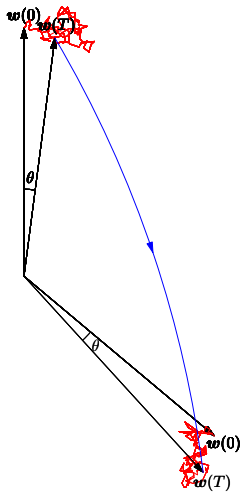
Skip Connections



- Breaks permutation symmetry. Used in Resnets, transformer, etc.

Symmetries

- The landscape potentially has a huge manifold with the same losses (neutral network)
- Find solution close to start
- We should think of optimisation more as a process of travelling than a process of arriving
- Of course even if we do minimise our loss we are not guaranteed to minimise our generalisation performance



Lessons

- SGD together with automatic differentiation has revolutionised machine learning
- There are a number of techniques to get the step size right: momentum (and Nesterov), adaptive step sizes (RMSProp), Adam and AdamW, learning-rate schedules
- Still need to understand that we are exploring a huge and complex loss landscape
- What works is problem specific (as always)